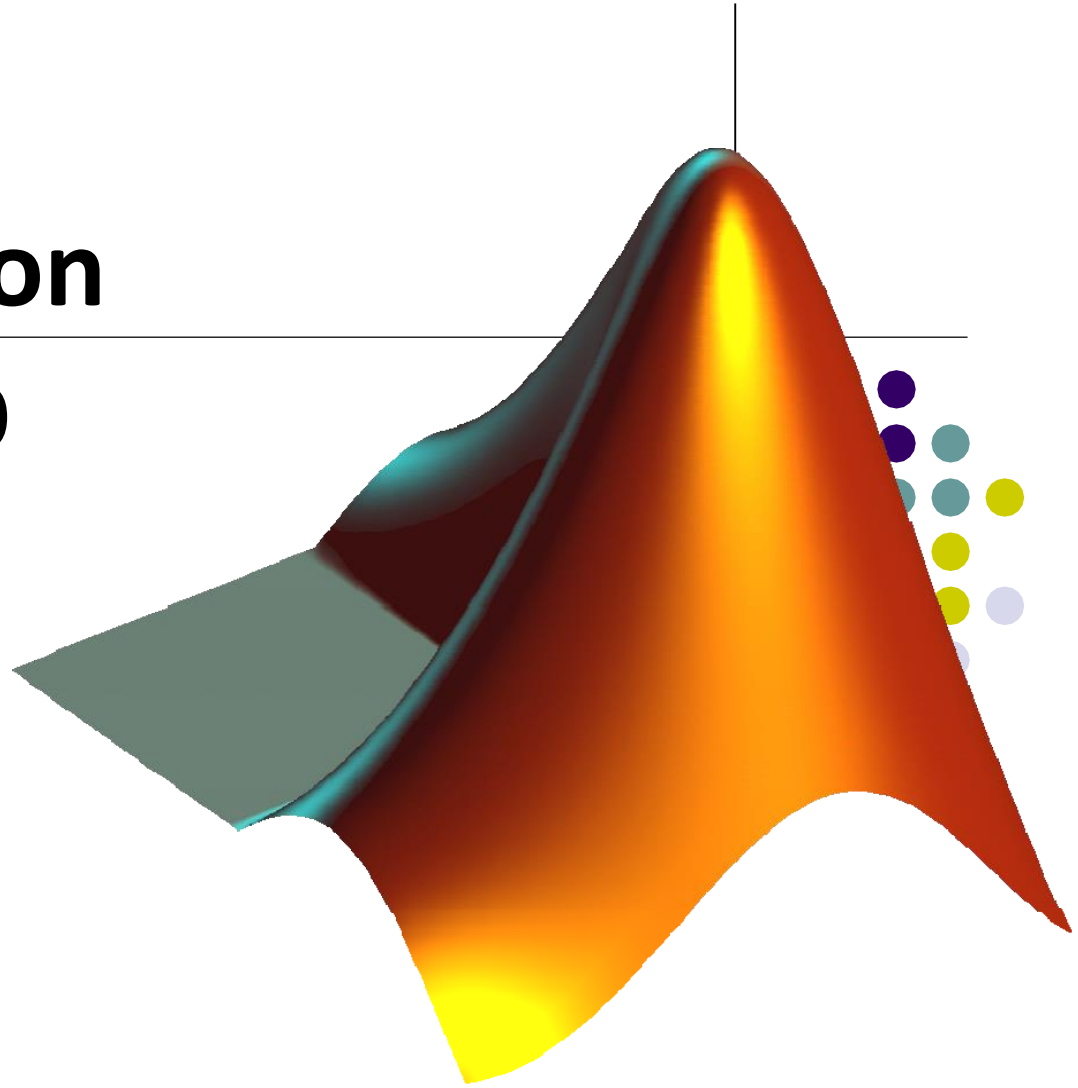
Introduction to Matlab



Outline

- Matlab introduction
- Matlab elements
  - Types
  - Variables
  - Matrices
- Loading, saving and ploting

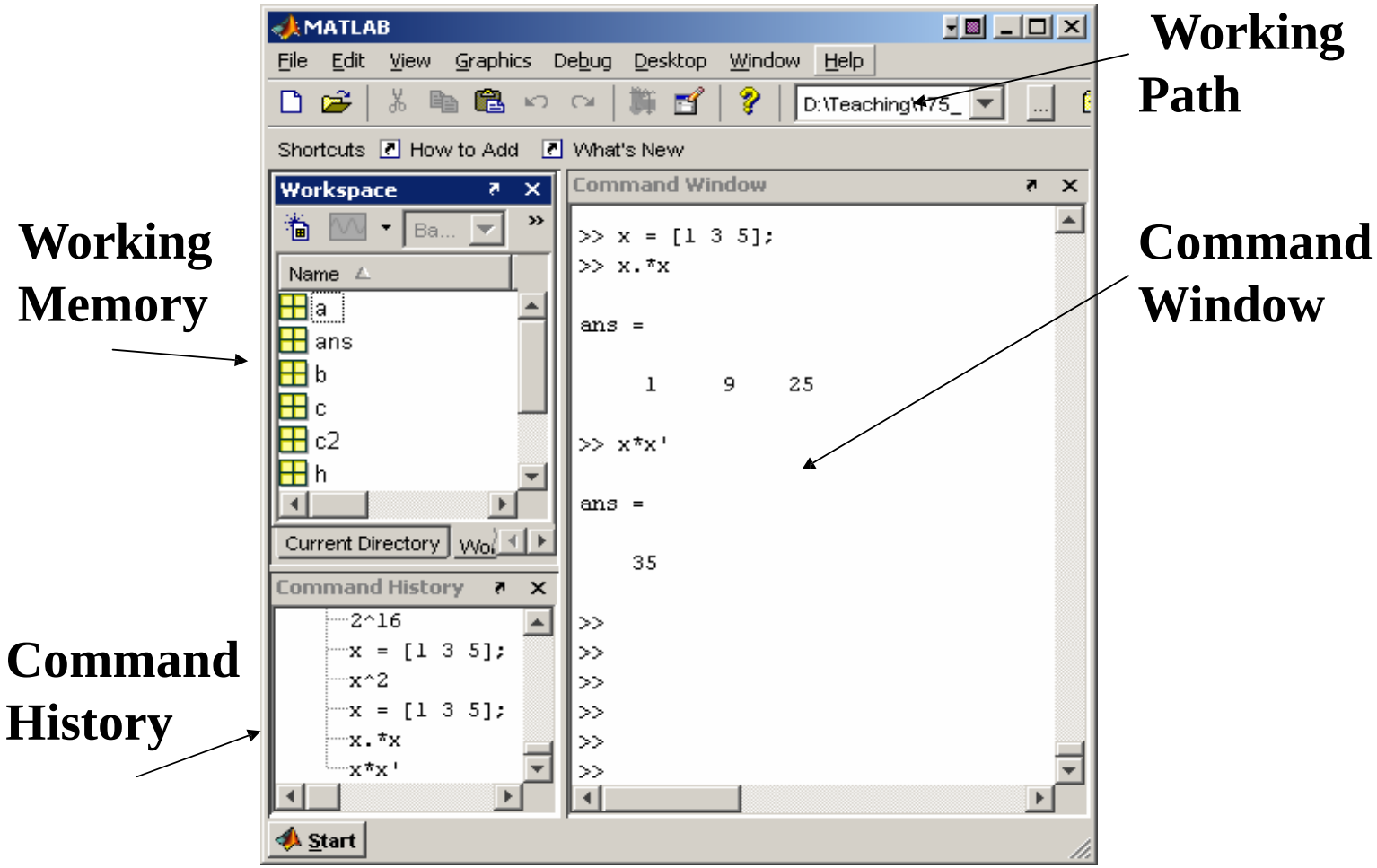
Matlab Introduction

- Matlab is a program for doing numerical computation. It's name is derived from MATrix LABoratory.
- Matlab is also a programming language that currently is widely used as a platform for developing tools for Machine Learning

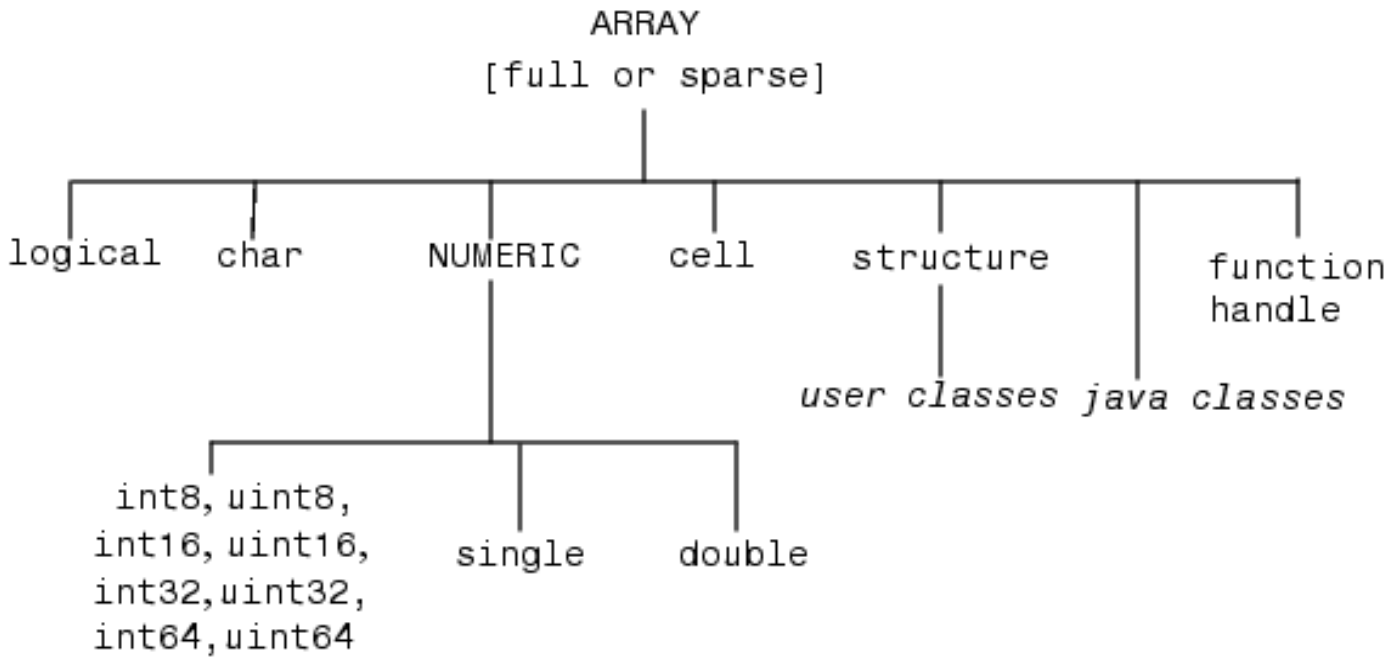
Matlab introduction

- Why it is useful for prototyping AI projects:
  - large toolbox of numeric/image library functions
  - very useful for displaying, visualizing data
  - high-level: focus on algorithm structure, not on low-level details
  - allows quick prototype development of algorithms

Opening Matlab



Data Types


ARRAY
[full or sparse]
logical
char
NUMERIC
cell
structure
function handle
user classes
java classes
int8, uint8, int16, uint16, int32, uint32, int64, uint64
single
double

Variables

- Have not to be previously declared
- Variable names can contain up to 63 characters
- Variable names must start with a letter followed by letters, digits, and underscores.
- Variable names are case sensitive

Matlab Assignment & Operators

Assignment	=	a = b (assign b to a)
Addition	+	a + b
Subtraction	–	a – b
Multiplication	* or .*	a*b or a.*b
Division	/ or ./	a/b or a./b
Power	^ or .^	a^b or a.^b

Matlab Matrices

- Matlab treats all variables as matrices. For our purposes a matrix can be thought of as an array, in fact, that is how it is stored.
- Vectors are special forms of matrices and contain only one row OR one column.
- Scalars are matrices with only one row AND one column

Matlab Matrices

- A matrix with only one row is called a row vector. A row vector can be created in Matlab as follows (note the commas):

```
» rowvec = [12,  14,  63]

rowvec =

    12    14    63
```

Matlab Matrices

- A matrix with only one column is called a column vector. A column vector can be created in MATLAB as follows (note the semicolons):

```
» colvec = [13 ; 45 ; -2]

colvec =

    13

    45

    -2
```

Matlab Matrices

- A matrix can be created in Matlab as follows (note the commas AND semicolons):

```
» matrix = [1 , 2 , 3 ; 4 , 5  ,6 ; 7 , 8 , 9]

matrix =

    1    2    3
    4    5    6
    7    8    9
```

Extracting a Sub-Matrix

- A portion of a matrix can be extracted and stored in a smaller matrix by specifying the names of both matrices and the rows and columns to extract. The syntax is:

```
sub_matrix = matrix ( r1 : r2 , c1 : c2 ) ;
```

where r1 and r2 specify the beginning and ending rows and c1 and c2 specify the beginning and ending columns to be extracted to make the new matrix.

Matlab Matrices

- A column vector can be extracted from a matrix. As an example we create a matrix below:

```
» matrix=[1,2,3;4,5,6;7,8,9]

matrix =
     1     2     3
     4     5     6
     7     8     9
```

- Here we extract column 2 of the matrix and make a column vector:

```
» col_two=matrix(1 :3 , 2)

col_two =
     2
     5
     8
```

Matlab Matrices

- A row vector can be extracted from a matrix. As an example we create a matrix below:

```
» matrix=[1,2,3;4,5,6;7,8,9]

matrix =
     1     2     3
     4     5     6
     7     8     9
```

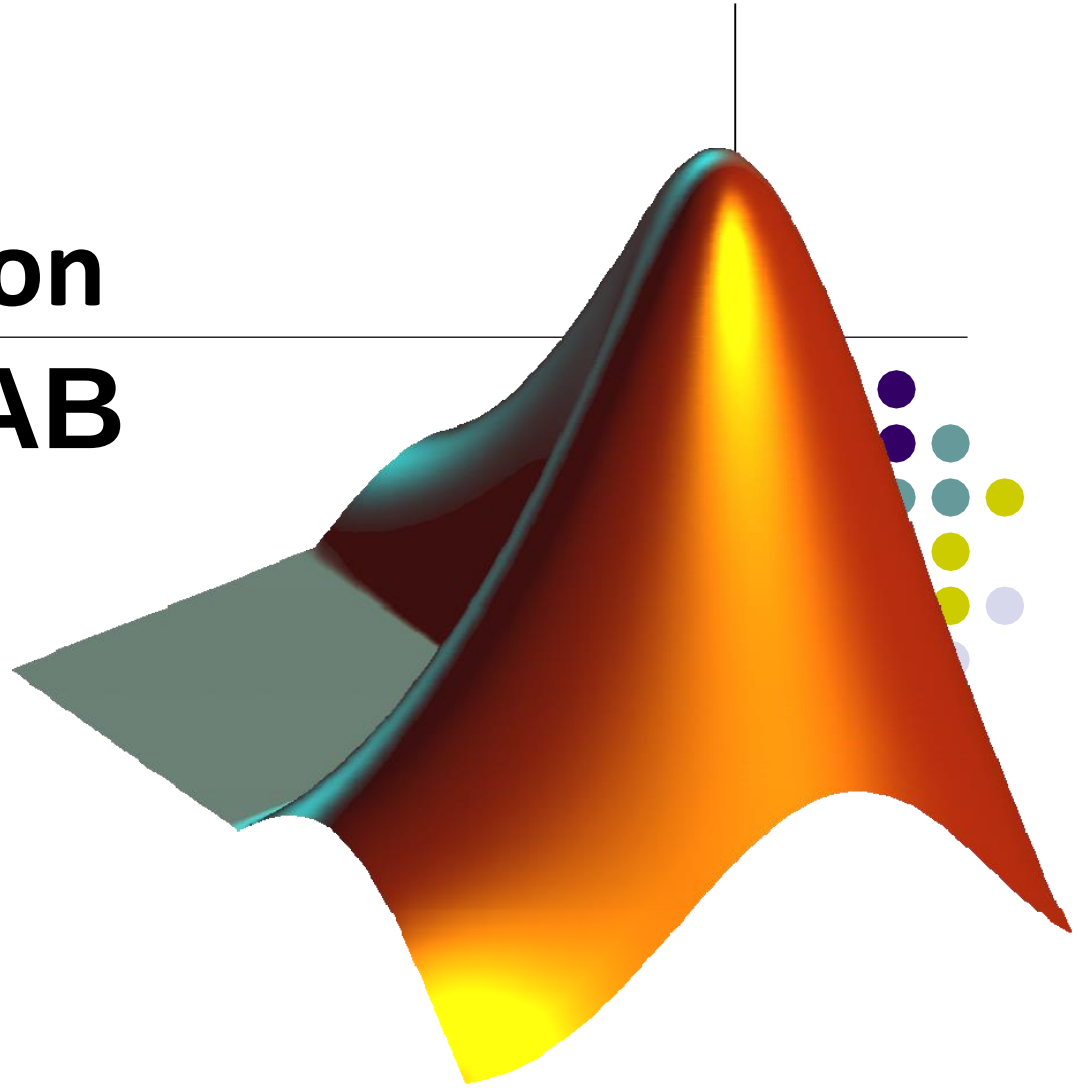
- Here we extract row 2 of the matrix and make a row vector. Note that the 2:2 specifies the second row and the 1:3 specifies which columns of the row.

```
» rowvec=matrix(2 : 2 , 1 : 3)

rowvec =
     4     5     6
```

Introduction to MATLAB

Part 2



Plotting with Matlab

- Matlab has a lot of function for plotting data. The basic one will plot one vector vs. another. The first one will be treated as the abscissa (or x) vector and the second as the ordinate (or y) vector. The vectors have to be the same length.

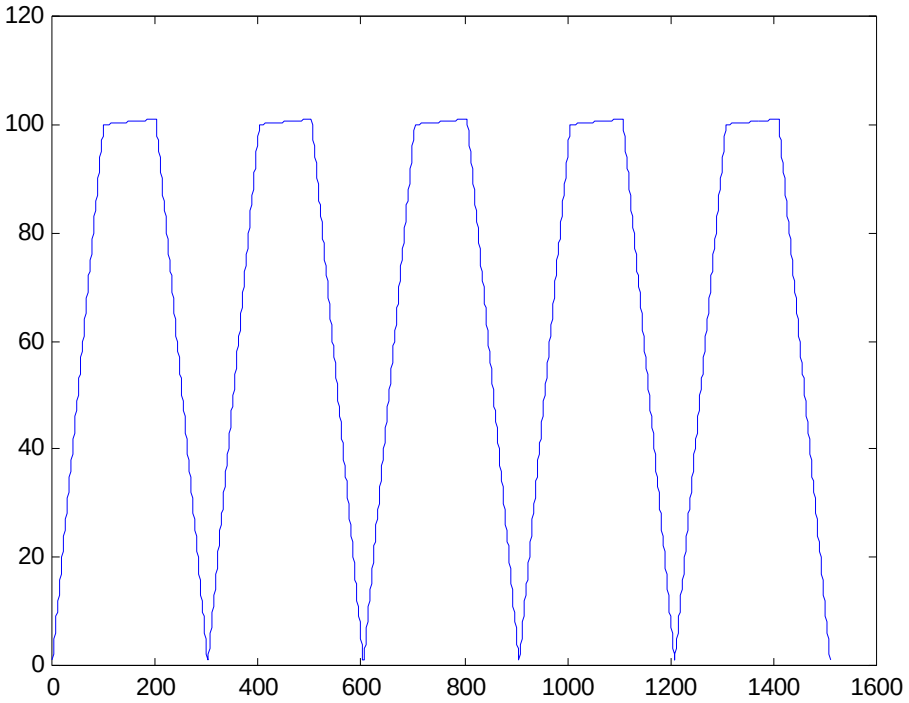
```
>> plot (time, dist)        % plotting versus time
```

- Matlab will also plot a vector vs. its own index. The index will be treated as the abscissa vector. Given a vector “time” and a vector “dist” we could say:

```
>> plot (dist)              % plotting versus index
```

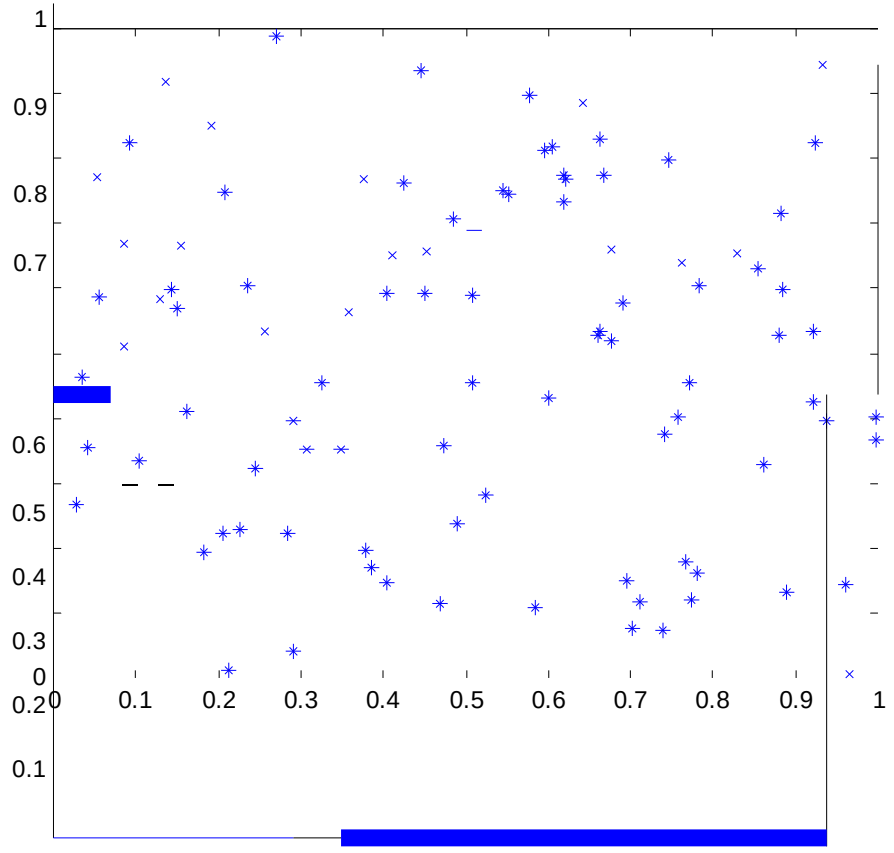
Plotting with Matlab

```
» a = 1:100;
» b = 100:0.01:101;
» c = 101:-1:1;
» d = [a b c];
» e = [d d d d d];
» plot(e)
```



Plotting with Matlab

```
» x = rand(1,100);
» y = rand(1,100);
» plot(x,y,'*')
```



Plotting with Matlab

- There are commands in Matlab to "annotate" a plot to put on axis labels, titles, and legends. For example:

```
>> % To put a label on the axes we would use:
>> xlabel ('X-axis label')
>> ylabel ('Y-axis label')

>> % To put a title on the plot, we would use:
>> title ('Title of my plot')
```

Matlab programming language

- Elements of Matlab as a programming language:
  - Expressions
  - Flow Control blocks
    - Conditional
    - Iterations
  - Functions

Expressions: Matlab Relational Operators

- MATLAB supports six relational operators.
  - Less Than <
  - Less Than or Equal <=
  - Greater Than >
  - Greater Than or Equal >=
  - Equal To ==
  - Not Equal To ~=

Expressions: Matlab Logical Operators

- MATLAB supports three logical operators.
  - not ~ % highest precedence
  - and & % equal precedence with or
  - or | % equal precedence with and

Expressions: Matlab Logical Functions

- MATLAB also supports some logical functions.

any (x)	returns 1 if any element of x is nonzero
all (x)	returns 1 if all elements of x are nonzero
isnan (x)	returns 1 at each NaN in x
isinf (x)	returns 1 at each infinity in x

Matlab Conditional Structures

```
if expression cond.
   sentences
elseif expr. cond.
   sentences
else
   sentences
end
```

```
a = input('value1? ');
b = input('value2? ');
if a == b,
   fprintf('a is equal to b\n');
elseif a > 0 && b > 0
   fprintf('both positive\n');
else
   fprintf('other case\n');
end
```

Matlab Iteration Structures (I)

```
for variable = expr
   sentence;
    ...
   sentence;
end
```

```
M = rand(4,4); suma = 0;
for i = 1:4
    for j = 1:4
            suma = suma + M(i,j);
    end
  end
fprintf('sum = %d\n',suma);
```

Matlab Iteration Structures (II)

```
while expr
   sentence;
    ...
   sentence;
end
```

```
M = rand(4,4);
i = 1; j = 1; suma = 0;

while i <= 4
  while j <= 4
    suma = suma + M(i,j);
    j = j+1;
  end
  i = i+1;
  j = 1;
end
fprintf('suma = %f\n',suma);
```